



Technische
Universität
Braunschweig

BACHELOR THESIS

Write down your Title right here!

Fabian Alich

5310894

Algorithmik

Institute of Operating Systems and Computer Networks
Algorithms Division

First reviewer

(Anon Anonymous, Institute of XY)

Second reviewer

(Anon Anonymous, Institute of YX)

Supervised by

(Anon Anonymous)

(tba)

22. Juli 2025

Statement of Originality

This thesis has been performed independently with the support of my supervisor/s. To the best of the author's knowledge, this thesis contains no material previously published or written by another person except where due reference is made in the text.

Braunschweig, 22. Juli 2025

Aufgabenstellung

This is where your “Aufgabenstellung” goes. You should get this from your supervisor!

Abstract

Your abstract gives a short introduction to your thesis (context), the problem description, and a brief description of your results.

You may want to include a german abstract, too.

Inhaltsverzeichnis

1	Introduction	1
1.1	Results	1
1.2	Related Work	1
1.2.1	Delaunay Triangulations	1
1.2.2	Flips in Triangulations	2
1.2.3	degree constrained minimum spanning tree	3
1.3	Overview	7
2	Preliminaries	9
3	Content	11
3.1	Instanzen Generation	11
3.1.1	Ja - Instanzen	11
3.1.2	Nein - Instanzen	12
3.2	Solver	13
3.2.1	Solver arten	13
3.2.2	Kanten Formulierung	13
3.2.3	Dreieck Formulierung	17
3.3	Heuristiken	17
3.3.1	Flips	18
3.3.2	OrTools mit Zielfunktion	18
3.4	Theoretische Mehrere Lösungen	19
4	Auswertung	21
4.1	Versuchsumgebung	21
4.2	Grund Solver (Eval1)	21
4.2.1	Dreiecke	21
4.2.2	Sat	22
4.2.3	OrTools	25
4.2.4	Gurobi	27
4.2.5	Gesamt	29
5	Conclusion (and Future Work)	31

Abbildungsverzeichnis

1.1	<i>A perspective view of a terrain</i> aus [4, p. 183]	2
1.2	<i>Examples of edge move, edge rotation, and edge slide</i> aus [1, p. 70]	3
3.1	Ein Vollständiger Graph mit allen möglichen Triangulation. Die Kanten e_2 und e_3 sind Schnittkanten von e_1	14
3.2	Ein Ausschnitt der Hülle	15
4.1	Enter Caption	22
4.2	Enter Caption	23
4.3	Enter Caption	25
4.4	Enter Caption	27
4.5	Enter Caption	28

Tabellenverzeichnis

1 Introduction

You can write an introduction here!

1.1 Results

We proof that the TRAVELING SALESMEN PROBLEM(TSP) is NP-complete.

... that the \textsc{Traveling Salesmen Problem}(TSP) is ...

You can use abbreviations for your problem names to avoid redundancy. Try to use existing abbreviations from related work if possible.

Formel zwischen lagern:

1.2 Related Work

In diesem Abschnitt werden verschiedene wissenschaftliche Arbeiten aus diesem Themenbereich betrachtet. Dies dient dazu, einen Überblick über den aktuellen Entwicklungsstand zu erhalten. Dazu werden drei Bereiche begutachtet. Der erste Bereich behandelt die *Delaunay Triangulation*. Diese zählt zu den bekanntesten Triangulations und bietet einen grundlegenden Einblick in das Thema Triangulation. Der zweite Bereich umfasst *Flips in Triangulations*. Hier wird untersucht, wie sich Triangulation durch lokale Änderungen gezielt modifizieren lassen. Der dritte und letzte Bereich befasst sich mit dem Problem des *degree constrained minimum spanning tree*. Dabei werden insbesondere Ansätze zur Einhaltung von Grad Beschränkungen betrachtet.

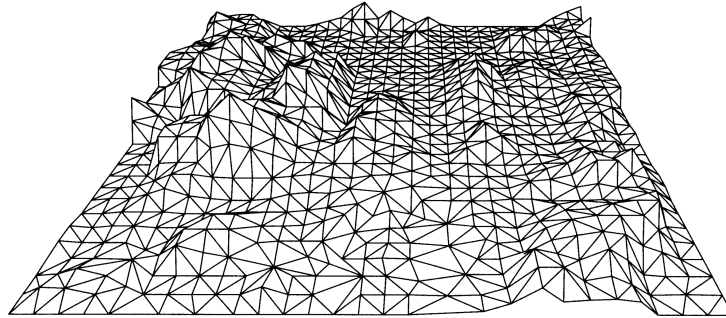
Es ist noch zu erwähnen, dass Sharir et. al. [12] den minimalen und maximalen Grad in einer Triangulation untersucht haben. Darüber hinaus haben Datta et. al. [2] und Gewali et. al. [7] sich mit Triangulation beschäftigt, bei denen jeder Knoten einen geraden Grad hat. Da diese beiden Themenbereiche keinen direkten Mehrwert für die vorliegende Arbeit bieten, werden sie hier lediglich der Vollständigkeitshalber namentlich erwähnt.

1.2.1 Delaunay Triangulations

Als Grundlage für den Algorithmus kann die *Delaunay-Triangulation* betrachtet werden. Diese liefert eine Triangulation, die möglichst gleichschenklige und gleichwinklige Dreiecke verwendet [11]. Dies wird erreicht, indem der minimale Winkel maximiert wird. Im Buch von de Berg et al. [4] wird diese Variante der Triangulation ausführlich behandelt.

Triangulations eignen sich durch ihre Struktur besonders gut zur Darstellung von Höhendaten im zweidimensionalen Raum (siehe Abbildung 1.1). Dabei kann ausgenutzt werden, dass Dreiecke unterschiedliche Innenwinkel und Kantenlängen besitzen.

Abbildung 1.1: *A perspective view of a terrain* aus [4, p. 183]



Das Grundproblem besteht darin, dass die Höhendaten nur an bestimmten Punkten bekannt sind und die restlichen Werte daher geschätzt werden müssen. Dies kann durch die Delaunay-Triangulation gut gelöst werden, da die Dreiecke aufgrund ihrer gleichschenkligen Eigenschaft visuell geeignet sind, die fehlenden Daten zu ergänzen.

Es werden auch Formel für die Anzahl der Kanten und Dreiecke in einer Triangulation angegeben. Für eine Punktmenge mit n Punkten und k Punkten auf der konvexen Hülle gilt:

$$\text{Anzahl Kanten: } 3n - 3 - k$$

$$\text{Anzahl Dreiecke: } 2n - 2 - k$$

1.2.2 Flips in Triangulations

Flips sind Möglichkeiten, um Kanten in einer Triangulation auszutauschen. Wie oben erwähnt, müssen Triangulation eine feste Anzahl an Kanten besitzen [4]. Dementsprechend können keine Kanten hinzugefügt oder entfernt werden, um den Grad einer Triangulation zu verändern. Daraus folgt, dass für jede entfernte Kante eine andere Kante eingefügt werden muss. Eine besondere Art dieser Umstrukturierung von Kanten sind sogenannte Flips. Hurtado et al. [8] zeigten, dass jede Triangulation mindestens

$$\frac{n - 4}{2}$$

flipbare Kanten besitzt, wobei n die Anzahl der Knoten bezeichnet. Laut der wissenschaftliche Arbeit *Flips in planar graphs* [1] lässt sich jede mögliche Triangulation in jede beliebige andere Triangulation überführen, indem die notwendigen Flips durchgeführt werden. Daraus folgt, dass, wenn eine Gradbeschränkte Triangulation möglich ist, diese mit den nötigen Flips konstruiert werden kann.

TODO überarbeiten **Sollte ich das später machen, hier die Sektion nennen**

Eine weitere Art von Flips sind sogenannte k -Flips. Diese stellen eine zusätzliche Möglichkeit dar, Kanten innerhalb einer Triangulation zu verschieben. Dabei werden k Kanten entfernt und durch k neue Kanten ersetzt. Normale Flips entsprechen somit

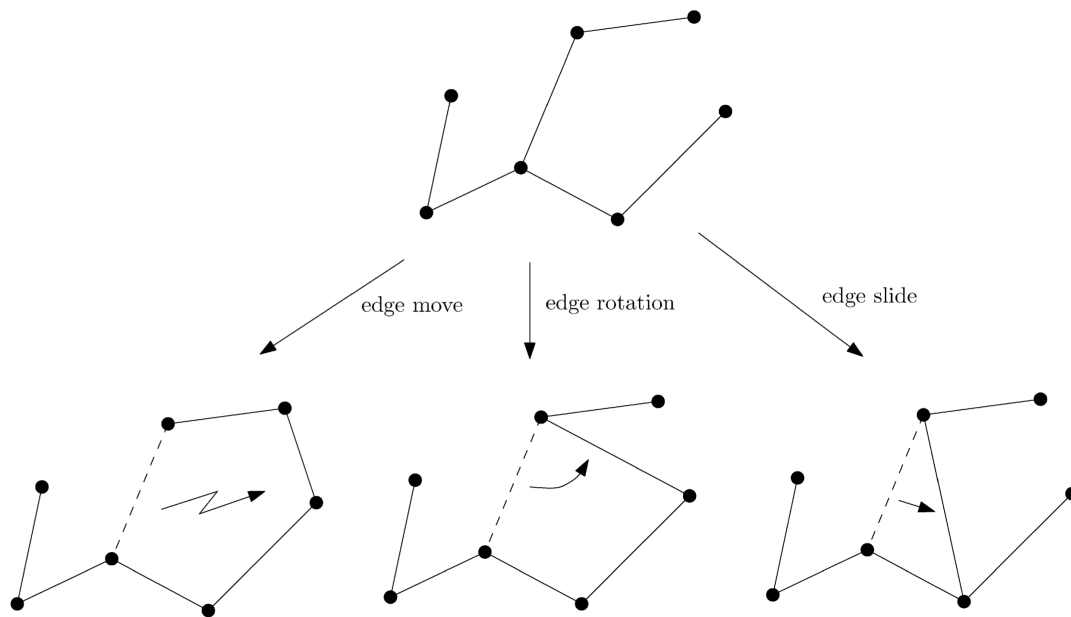


Abbildung 1.2: *Examples of edge move, edge rotation, and edge slide* aus [1, p. 70]

1-Flips. Der Vorteil von k -Flips liegt darin, dass mit einem Schritt größere strukturelle Veränderungen möglich sind.

Drei beispielhafte Flips sind in Abbildung 1.2 dargestellt. Zum einen der *edge move*, hierbei handelt es sich um das einfache Entfernen und anschließende Einfügen einer Kante. Etwas stärker eingeschränkt ist die *edge rotation*, bei der ein Endpunkt der Kante unverändert bleibt. Die Rotation erfolgt also nur um einen Knoten. Die dritte Variante, der *edge slide*, schränkt die *edge rotation* weiter ein. Hier darf der veränderte Knoten nur ein Nachbar des ursprünglichen Knotens sein.

1.2.3 degree constrained minimum spanning tree

Eine Gradbeschränkung gibt es auch beim Minimum Spanning Tree. Hier hat jede Kante ein vorgegebenes Gewicht, welches im Spanning Tree minimiert werden soll. Das Besondere am *degree constrained minimum spanning tree* ist, dass ein Minimum Spanning Tree erzeugt werden soll, welcher an jedem Knoten einen maximalen Grad von d hat. Dabei wird d global für das Problem vorgegeben.

Es wurden verschiedene Ansätze von Joshua Knowles und David Corne [9] vorgestellt, welche das Problem lösen sollen. Der erste Ansatz ignoriert zu Beginn den Aspekt der Minimalität des Problems. Es werden Kanten hinzugefügt, welche die Gradbeschränkung nicht verletzen. Im weiteren Verlauf wird versucht, die Anzahl der Kanten zu minimieren.

Ein anderer Ansatz nutzt aus, dass der minimale Spannbaum ohne Grad Beschränkung in polynomialer Zeit gelöst werden kann. Dieser Ansatz erzeugt zuerst einen minimalen

1 Introduction

Spannbaum, welcher die Gradbeschränkung ignoriert. Wenn eine Lösung gefunden wurde, werden den Kanten neue Gewichte zugeteilt. Diese neuen Gewichte werden mit folgender Formel berechnet,

$$weight + fault \cdot max \cdot \frac{(weight - min)}{(max - min)}$$

wobei *weight* das aktuelle Gewicht der Kante ist. Die Variable *fault* hat den Wert, $\{0, 1, 2\}$ abhängig davon, wie viele Knoten an der Kante gerade die Gradbeschränkung verletzen. Die Variablen *min* und *max* bezeichnen das kleinste beziehungsweise größte Gewicht aller Kanten. Danach wird erneut ein minimaler Spannbaum gebildet, wobei durch das neue Gewicht nun Kanten bevorzugt werden, welche keine Gradbeschränkung verletzen. Diese beiden Schritte können so lange wiederholt werden, bis eine valide Lösung gefunden wurde oder eine maximale Anzahl an Iterationen erreicht wurde. Das Problem bei diesem Ansatz ist, dass Lösungen fälschlicherweise als *nicht möglich* bewertet werden können aufgrund der maximalen Iterationen.

Ein weiterer Ansatz wird als *Hillclimbing* bezeichnet. Dieser löst das Problem des *degree constrained minimum spanning tree* nicht, wird jedoch zur Suche nach neuen Knoten in einem anderen Algorithmus verwendet. In diesem Ansatz werden lokal neue Knoten gesucht, die bessere Ergebnisse liefern. Dies könnte auf die *Degree constrained Triangulation* angewendet werden, um neue Kanten zu finden. Der Ansatz erzeugt keine eigene Lösung, sondern verbessert eine bestehende. Zunächst wird ein zufälliger Knoten aus einer Lösung ausgewählt. Danach werden weitere mögliche Knoten in der Nähe des ursprünglichen Knotens gesucht. Bei diesen Knoten wird überprüft, ob sie gleich gute oder bessere Ergebnisse liefern. Falls dies der Fall ist, wird der Knoten ausgetauscht, andernfalls wird der gefundene Knoten ignoriert. Dieser Ansatz soll lokal gute Ergebnisse liefern, bringt jedoch aufgrund seines *greedy*-Verhaltens keine globalen Verbesserungen. Eine mögliche Verbesserung der lokalen Problematik stellt *multistart hillclimbing* dar. Bei diesem Verfahren wird nur ein Punkt in der Nähe getestet. Ist dieser nicht gleich oder besser, wird ein neuer zufälliger Punkt ausgewählt.

Der nächste Ansatz wird *simulated annealing* genannt. Auch hier werden nur neue Knoten gesucht, um bestehende Lösungen zu verbessern. Das Ziel des Ansatzes ist es, das Problem mit einer globalen Verbesserung zu lösen, indem zu Beginn viele Veränderungen zugelassen werden und im Verlauf immer genauere Verbesserungen erfolgen. Auch hier wird ein zufälliger Knoten ausgewählt und es werden weitere mögliche Knoten in der Nähe gesucht. Bei diesem Ansatz werden jedoch neue Knoten gemäß der folgenden Formel übernommen:

$$p\{\text{accept } j\} = \begin{cases} 1 & \text{if } f(j) \leq f(i) \\ \exp\left(\frac{f(i)-f(j)}{c}\right) & \text{sonst} \end{cases}$$

Dabei ist $p\{\text{accept } j\}$ die Wahrscheinlichkeit, dass ein Knoten übernommen wird. Die aktuelle Lösung ist i und die mögliche neue Lösung ist j . Die Funktion f gibt hierbei die Kosten der Lösung an. Der Parameter c wird zu Beginn sehr hoch gesetzt und mit der Zeit verringert. Die Veränderung von c bewirkt die gewünschte Änderungsrate. Der Ansatz endet, wenn ein finales c_f erreicht wird, also wenn $c = c_f$.

Der letzte Ansatz wird erst am Ende vorgestellt und beschreibt das übliche Verfahren eines Evolutionsalgorithmus. Dabei werden verschiedene Lösungen erzeugt und bewertet. Die besten Lösungen werden ausgewählt und auf Basis dieser Lösungen werden durch Mutation neue Lösungen generiert.

Krishnamoorthy et.al. [10] hat Evolutionsalgorithmen noch mal genauer betrachtet. Der erste Schritt ist es eine Codierung zu wählen, welche nur *richtige* Spannbäume darstellen kann. Sonst kann es passieren, dass von den erstellten Lösungen viele nicht genutzt werden können und somit nicht relevant sind. Wenn eine passende Codierung gewählt wurde, können dann nur noch der Grad und das Gewicht betrachtet werden.

Eine Möglichkeit wie Mutationen stattfinden können, ist die *Crossover* Methode. Dabei existieren zwei *Eltern* Lösungen P_1 und P_2 . Aus diesen werden

$$2 \cdot (n - 2)$$

verschiedene *Kinder* erzeugt, da es $n - 1$ Veränderungsmöglichkeiten gibt. Anschließend werden die *Kinder* bewertet und sortiert. Das beste *Kind*, das P_1 am ähnlichsten ist, wird ausgewählt. Ist dieses *Kind* besser als P_1 , ersetzt es P_1 . Für P_2 wird ein *Kind* nach dem gleichen Verfahren ausgewählt.

Eine weitere Möglichkeit ist die *Breeding* Methode. Dabei existiert ein Pool aus Lösungen. Zufällig werden Paare gebildet, die jeweils zwei *Kinder* erzeugen. Von diesen *Kindern* ersetzt das bessere *Kind* das schlechtere *Elternteil*. Die Größe des Pools kann so angepasst werden, dass alle möglichen Veränderungen berücksichtigt werden.

Beide Methoden sollen im Durchschnitt alle 10 Generationen die beste Lösung mit allen anderen Lösungen verglichen haben. Als mögliche Iterationsanzahl wird folgende Formel verwendet,

$$\frac{50 \cdot n}{\bar{b}}$$

dabei bezeichnet $\bar{b}$ die durchschnittliche Gradbeschränkung. Diese Formel sorgt dafür, dass große Instanzen mehr Iterationen erhalten, während leichtere Instanzen (mit höherem Grad) schneller bearbeitet werden.

Sandor P. Fekete et al [6] betrachten ebenfalls das *degree constrained minimum spanning tree* problem. Hierbei wird ein *Flow based Algorithmus* verwendet. Da dieser Ansatz wahrscheinlich nicht gut auf Triangulations übertragbar ist, wird er hier nur kurz behandelt.

Im praktischen Teil des Papers werden zwei Algorithmen vorgestellt. Diese erhalten einen minimalen Spannbaum und verändern ihn so, dass er eine Gradbeschränkung einhält. Dabei wird darauf geachtet, dass sich das Gewicht des Spannbaums nicht stark verschlechtert. Für diese Algorithmen werden Hauptoperationen namens *Adoptions* eingeführt. Diese funktionieren ähnlich wie Flips nur für einen Spannbaum. Es handelt sich also um Operationen, die eine legale Veränderung bewirken. Die beiden Algorithmen liefern eine Liste von Adoptionen, welche die gewünschte Änderung erzeugen. Der erste Algorithmus nutzt eine fraktionale Lösung und eine Greedy Strategie, um die Adoptionen zu finden. Der andere Algorithmus beschränkt die betrachteten Kanten auf jene des gegebenen Spannbaums. So können die Algorithmen in polynomialer Zeit eine Lösung finden, die eine Gradbeschränkung einhält.

1 Introduction

Eine Abwandlung des minimalen Spannbaums mit Gradbeschränkung wurde von Ana Maria de Almeida et al. [3] untersucht. Dabei erweitern sie das Problem um eine Funktion, welche für jeden Knoten einen Mindestgrad vorgibt. Dieses Problem wird als *Min-Degree Constrained Minimum Spanning Tree* bezeichnet.

Es wurde das *k-Dimensional Matching Problem* verwendet, um zu zeigen, dass das *Min-Degree Constrained Minimum Spanning Tree* Problem in NP liegt. Zu erwähnen ist, dass dies in der dritten Dimension nicht gezeigt wurde. Allerdings soll das Problem schwieriger sein als das *Degree Constrained Minimum Spanning Tree*, obwohl auch dieses in NP liegt.

Auch in diesem Fall wird das Problem mit einer *Flow based Formulierung* betrachtet. Dabei werden zwei Formulierungen angegeben. Eine gerichtete und eine ungerichtete Formulierung. Die gerichtete Formulierung soll hierbei effizienter Ergebnisse liefern. Die Formulierungen wurden mit einem *Branch and Bound* Algorithmus getestet. Dabei wurde festgestellt, dass es bei der gerichteten Formulierung relevant ist, welcher Knoten als Wurzelknoten ausgewählt wird. Es konnte jedoch kein einheitlich optimaler Wurzelknoten für die Formulierungen festgestellt werden. Des Weiteren wurde häufig festgestellt, dass die Relaxierung keine guten Schranken lieferte. Dies konnte damit erklärt werden, dass die zugehörige Formel für den minimalen Spannbaum ohne Gradbeschränkung gleiche Ergebnisse lieferte.

1.3 Overview

If needed, write down where to find everything.

2 Preliminaries

Definition 2.1 (Überschneidung). Zwei Kanten gelten als überschneidend, wenn sie sich in der Ebene kreuzen. Im Folgenden bezeichnet die Funktion $I : (\mathbb{Q}^2 \times \mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2 \times \mathbb{Q}^2)$ die Menge aller Kanten, die eine gegebene Kante schneiden.

Definition 2.2 (Triangulation). Eine Triangulation einer Punktmenge $P \subset \mathbb{Q}^2$ ist ein maximaler kreuzungsfreier Graph auf P . Das bedeutet, es können keine weiteren Kanten hinzugefügt werden, ohne dass sich Kanten schneiden.

Definition 2.3 (Grad). Der Grad eines Knotens ist die Anzahl der anliegenden Kanten. Im Folgenden wird er durch die Funktion $\deg : \mathbb{Q}^2 \rightarrow \mathbb{N}$ angegeben.

Definition 2.4 (Gradbeschränkte Triangulation). Die Gradbeschränkte Triangulation hat die gleichen Voraussetzungen wie eine Triangulation. Zusätzlich gibt es noch eine Funktion $\deg^* : \mathbb{Q}^2 \rightarrow \mathbb{Q}$. Diese gibt den Grad an welcher ein Knoten haben muss.

Definition 2.5 (Flip). Ein Flip ist eine Operation, bei der eine Kante entfernt und eine andere hinzugefügt wird, sodass die Eigenschaften einer Triangulation erhalten bleiben. In dieser Arbeit wird nur der Diagonale Flip betrachtet, bei dem die Diagonale eines konvexen Vierecks durch die andere Diagonale ersetzt wird. Ein Flip wird durch die Funktion $\text{flip} : (\mathbb{Q}^2 \times \mathbb{Q}^2) \rightarrow (\mathbb{Q}^2 \times \mathbb{Q}^2)$ definiert.

Definition 2.6 (Konvexe Hülle mit Randpunkten). Die konvexe Hülle einer Punktmenge ist die kleinste konvexe Menge, die alle Punkte enthält. Die Randpunkte sind die Punkte, die auf dem Rand der konvexen Hülle liegen, einschließlich solcher, die nicht an einer Ecke liegen. Die konvexe Hülle inklusive Randpunkte wird durch die Funktion $\text{CH}^* : \mathcal{P}(\mathbb{Q}^2) \rightarrow \mathcal{P}(\mathbb{Q}^2)$ definiert.

3 Content

3.1 Instanzen Generation

3.1.1 Ja - Instanzen

Bei den Ja-Instanzen handelt es sich um Knotenmengen, bei denen die $\deg^*$ -Funktion Grade liefert, welche eine gradbeschränkte Triangulation ermöglichen. Hierbei ist nicht bekannt, wie viele mögliche Lösungen existieren, sondern nur, dass es mindestens eine gibt.

Der generelle Ablauf zur Erzeugung von Ja-Instanzen beginnt mit einer zufälligen Knotenmenge. Dabei wird lediglich die Anzahl der Knoten vorgegeben. Anschließend wird mit einem klassischen Triangulationsverfahren eine initiale Triangulation erstellt. Von dieser werden dann die Grade und die Knotenkoordinaten gespeichert, um sie als Eingabe für die zu testenden Algorithmen zu verwenden.

Delaunay

Das erste klassische Verfahren zur Gradbestimmung verwendet die Delaunay-Triangulation. Die Delaunay-Triangulation wurde bereits im Kapitel Abschnitt 1.2.1 vorgestellt. Für die Erstellung der Delaunay-Triangulation wird die Bibliothek `scipy` genutzt.

Delaunay mit zufälligen Flips

Das zweite Verfahren zur Erzeugung von Ja-Instanzen basiert auf der Delaunay-Triangulation. zusätzlich werden jedoch zufällige Flips durchgeführt. Zunächst wird eine Delaunay-Triangulation der Knotenmenge erstellt. Auf dieser Triangulation werden anschließend k zufällige Flips gemäß Definition 2.5 durchgeführt. Für jeden Flip wird zufällig eine Kante e_0 ausgewählt. Für diese Kante werden zunächst alle Dreiecke gesucht, die diese Kante enthalten. Dies geschieht, indem für die beiden Knoten $x_{0,0}$ und $x_{0,1}$ der Kante e_0 alle ausgehenden Kanten verglichen werden. Sollten zwei dieser Kanten den gleichen Knoten v haben, muss das Tripel $(x_{0,0}, x_{0,1}, v)$ ein Dreieck bilden, welches die Kante e_0 enthält.

Sollten mehr als zwei Dreiecke gefunden werden, so müssen die beiden Dreiecke ausgewählt werden, welche leer sind (keine weiteren Knoten enthalten). Da es in einer Triangulation nicht zu Überschneidungen kommen darf, kann es nur zwei leere Dreiecke geben. Aus diesen beiden Dreiecken $(x_{0,0}, x_{0,1}, v_0)$ und $(x_{0,0}, x_{0,1}, v_1)$ wird dann die Kante e_0 entfernt und durch die neue Kante, bestehend aus v_0 und v_1 , ersetzt.

Dieser Prozess verändert die lokale Struktur der Triangulation und erzeugt so Knotenmengen mit unterschiedlichen Gradverteilungen, behält aber die Triangulationseigenschaft bei.

Zufällige Kantenerzeugung

Das nächste Verfahren basiert auf der zufälligen Auswahl von Kanten. Hierbei werden zunächst alle geometrisch möglichen Kanten zwischen den Punkten identifiziert. In diesem Schritt werden Kanten ausgeschlossen, welche durch andere Knoten der Knotenmenge verlaufen würden.

Die verbleibenden gültigen Kanten werden dann in einer zufälligen Reihenfolge betrachtet. Bei der Auswahl der finalen Kanten wird für jede Kandidatenkante geprüft, ob sie eine der bereits ausgewählten Kanten schneidet. Nur wenn keine Schnitte entstehen, wird die Kante der Triangulation hinzugefügt.

Dieses Verfahren ist maximal da alle möglichen Kanten betrachtet wurden.

Iteration

Ein weiteres Verfahren zur Erzeugung von Ja-Instanzen basiert auf der Iteration. Hierbei werden zunächst alle möglichen Kanten zwischen den Punkten identifiziert. Diese werden nun nacheinander betrachtet und wenn sie keine andere Kante schneiden hinzugefügt. Diese Verfahren ist ebenfalls maximal, da alle möglichen Kanten betrachtet wurden. Beim Iterativen Verfahren ist zu beachten das Triangulationen erzeugt werden welche einen Knoten haben welcher einen sehr hohen Sollgrad hat. Dies basiert darauf das die Kanten in der Reihenfolge betrachtet werden wie sie erstellt werden. Das hat den Effekt das die alle Kanten des ersten Knoten hinzugefügt werden können da es noch keine anderen Kanten gibt welche diese schneiden könnten.

Greedy

Das letzte Verfahren ist das Greedy Verfahren. Hierbei werden auch alle möglichen Kanten zwischen den Punkten identifiziert. Allerdings werden sie im gegensatz zum Iterativen Verfahren zunächst nach der Länge sortiert. Hierbei ist es möglich mit der kürzesten oder der längsten Kante zu beginnen. Anschließend wird wie beim Iterativen Verfahren jede Mögliche Kante nacheinander hinzugefügt.

3.1.2 Nein - Instanzen

Nein Instanzen sind Knotenmengen, bei denen die $\deg^*$ Funktion Gradwerte liefert, die keine gradbeschränkte Triangulation zulassen. Diese Instanzen entstehen nicht direkt, sondern werden aus Ja Instanzen abgeleitet. Nach der Generierung muss zusätzlich geprüft werden, ob die Instanz nicht möglich ist. Da die beschriebenen Verfahren nur mit hoher Wahrscheinlichkeit Nein Instanzen erzeugen. Es ist zu beachten, dass die Summe der Sollgrade mithilfe der Polyederformel einfach berechnet und überprüft werden kann. Daher besitzen die abgeleiteten Nein Instanzen insgesamt die gleichen Sollgrade wie die Ja Instanzen. Zusätzlich darf kein Sollgrad größer als die Anzahl der Knoten abzüglich eins sein, da sonst der Sollgrad nicht erfüllt werden kann da nicht genügend Kanten erzeugt werden können.

Bei der ersten Methode werden zwei Knoten ausgewählt. Vom ersten Knoten wird ein Wert abgezogen und beim zweiten Knoten derselbe Wert hinzugefügt. Zunächst wird eine zufällige Zahl r zwischen null und c gewählt. Hierbei ist c eine konstante Zahl, die beim Generieren festgelegt wird. Anschließend werden zwei Knoten v_1 und v_2 ausgewählt. Dabei müssen folgende Bedingungen erfüllt sein

$$v_1 \neq v_2 \quad (3.1)$$

$$\deg^*(v_1) < r + 2 \quad (3.2)$$

$$\deg^*(v_2) + r > n - 1 \quad (3.3)$$

wobei n die Anzahl der Knoten ist. Wenn beide Knoten die Bedingungen erfüllen, wird der Sollgrad von v_1 um r verringert. Der Sollgrad von v_2 wird um r erhöht.

Die zweite Methode besteht darin, zwei Knoten auszuwählen und deren Sollgrade zu tauschen. Es muss lediglich beachtet werden, dass es sich um zwei verschiedene Knoten handelt.

Beide Ansätze werden mehrfach angewendet, um die Wahrscheinlichkeit zu erhöhen, dass die Instanzen nicht mehr möglich sind. Knoten werden dabei nicht mehrfach betrachtet.

3.2 Solver

3.2.1 Solver arten

- PySat
- OrTools
- Gurobi

3.2.2 Kanten Formulierung

Es gibt zwei notwendige Beschränkungen für eine gradbeschränkte Triangulation. Die erste Beschränkung ist die Überschneidungsbeschränkung. Eine Triangulation darf keine Überschneidungen enthalten. Dafür werden alle Kantenpaare berechnet, die sich schneiden. Von jedem dieser Paare darf nur eine Kante aktiv sein. TODO Algorithmus erklären oder darauf verweisen. Die Schnittpaare werden mit dem Algorithmus berechnet.

Die zweite Beschränkung ist die Gradbeschränkung. Für jeden Knoten i wird die Summe der ausgehenden Kanten auf den Sollgrad $\deg^*(i)$ beschränkt. Um dies zu erreichen, wird für jeden Knoten festgelegt, dass die Summe der ausgehenden Kanten dem Sollgrad entspricht.

Eine weitere notwendige Bedingung ist, dass die Kantenanzahl maximal ist. Dies muss nicht explizit als Beschränkung angegeben werden da es von der Gradbeschränkung impliziert wird. Es muss nur geprüft werden, ob die Gradbeschränkung in einer Triangulation erfüllbar ist. Dafür kann die Formel

$$\text{Anzahl Kanten} = 3n - 3 - k$$

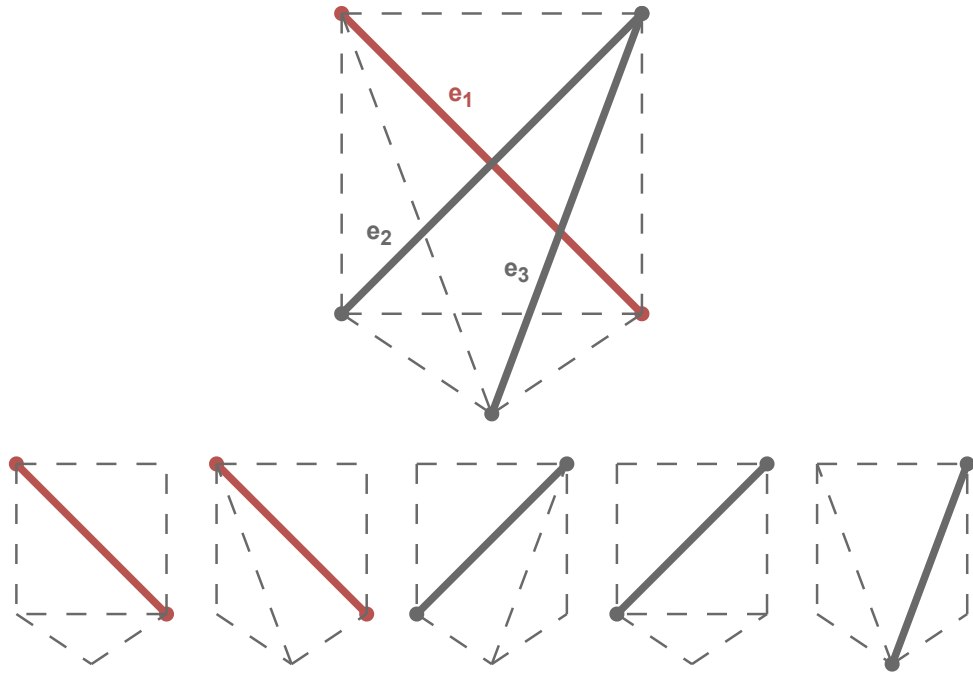


Abbildung 3.1: Ein Vollständiger Graph mit allen möglichen Triangulation. Die Kanten e_2 und e_3 sind Schnittkanten von e_1 .

aus Abschnitt 1.2.1 verwendet werden. Die Anzahl der Kanten kann in folgender Formel genutzt werden, um die Gradbeschränkung zu überprüfen.

$$\text{Anzahl Kanten} \cdot 2 \stackrel{!}{=} \sum \deg^*(i)$$

Alle weiteren Beschränkungen sind optional. Sie sind Möglichkeiten, die Laufzeit des Solvers zu verbessern. Die erste und naheliegende Beschränkung ist die *fixierte Hülle* Beschränkung. Hierbei werden die Kanten der konvexen Hülle immer hinzugefügt. Diese dürfen hinzugefügt werden, da sie keine Schnittkanten besitzen. Sie dürfen auch die Gradbeschränkung nicht verletzen. Andernfalls wäre der Graph nicht maximal und somit keine Triangulation.

Eine andere mögliche Beschränkung ist die *alternative Kanten* Beschränkung. Diese wurde von Sándor P. Fekete et.al. [5] übernommen. Diese Beschränkung nutzt aus das der Graph maximal sein muss. Damit dies der Fall sein kann muss für jede Kante gelten das entweder die Kante aktiv ist oder eine der Schnittkanten aktiv ist. Wie man in Abschnitt 3.2.2 sieht, ist e_1 die ausgewählte Kante und e_2 und e_3 die Schnittkanten. Weiter unten im Bild ist zu sehen das alle möglichen Triangulation mindestens eine der drei Kanten verwendet. Zu beachten ist allerdings das die Anzahl der Schnittkanten nicht gleich eins sein darf. Nur mindestens eine sollte die Kante nicht aktiv sein. Wie man im Bild sieht können die Kanten e_2 und e_3 gleichzeitig aktiv sein.

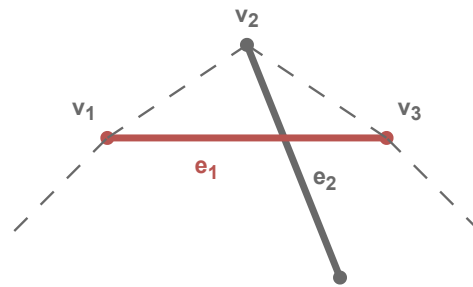


Abbildung 3.2: Ein Ausschnitt der Hülle

Eine weitere optionale Beschränkung ist zuvor berechnete Kanten zu fixieren oder auszuschließen. Eine Möglichkeit um Kanten zu fixieren ist es immer drei aufeinander folgende Knoten der Hülle zu betrachten. Ein mögliches Beispiel ist in Abschnitt 3.2.2 zu sehen. Sollte $\deg^*(v_2) = 2$ sein muss die Kante e_1 in der Triangulation genutzt werden. Dies hat den Grund das alle Schnittkanten von e_1 nicht aktiv sein können. Sie würden sonst den Grad von v_2 erhöhen. Analog dazu kann die Kante e_1 ausgeschlossen werden sollte $\deg^*(v_2) > 2$. Sonst kann der Sollgrad nicht erreicht werden.

TODO genau formal nochmal klären für die Kanten ausschließen.

Die nächste Bedingung ist nur theoretischer Test. Mit den bisher genannten Beschränkungen können nicht viele Kanten ausgeschlossen werden. Es kann also nicht betrachtet werden welchen Einfluss große Mengen von ausgeschlossenen oder fixierten Kanten auf die Laufzeit haben. Daher werden für einzelne Instanzen Kanten berechnet welche sicher ausgeschlossen oder fixiert werden können. Die Idee ist es zunächst die Instanz mit einem beliebigen Lösungsansatz zu lösen. Von diesen gefundenen Kanten wird jetzt ein Prozentsatz zufällig ausgewählt. Diese Kanten können sicher fixiert werden. Für ausschließbare Kanten werden alle Kanten betrachtet welche nicht genutzt wurden. Von diesen Kanten wird ebenfalls ein zufälliger Prozentsatz ausgewählt. Diese ausgewählten Kanten werden nacheinander bei einem Solver verboten. Sollte der Solver keine Lösung mehr finden, wird die Kante als ausschließbar markiert.

- Exclude Edges
 - Knoten mengen halbieren, gucken ob Gradbeschränkung erfüllbar ist
 - [https://www.notion.so/Fragen-2-20eb105ff608808895e5fd5389915074?](https://www.notion.so/Fragen-2-20eb105ff608808895e5fd5389915074?source=copy_link#216b105ff60880c1bc3ce905e0bd8d94)
source=copy_link#216b105ff60880c1bc3ce905e0bd8d94

Für alle Solver wird angenommen das E die Menge aller möglicher Kanten sei. Also Kanten die keine Punkte schneiden. Für jede Kante $e \in E$ wird eine Bool Variable x_e erstellt. Diese Variable ist wahr wenn die Kante e in der Triangulation enthalten ist.

OrTools und Gurobi

Für ein exaktes Ergebnis kann sowohl OrTools als auch Gurobi verwendet werden. Da beide Solver eine ähnliche Art der Formulierung verwenden werden sie hier zusammengefasst. TODO Reinforme geändert Die Überschneidungsbeschränkung wird wie in Abschnitt 3.3.2 implementiert. Gurobi nutzt hier äquivalent:

$$\text{model.addConstr}(x_{e_1} + x_{e_2} \leq 1)$$

Für die Gradbeschränkung wird die Summe der booleschen Variablen x_e für alle ausgehenden Kanten eines Knotens i auf den Sollgrad $\deg^*(i)$ beschränkt.

Für die Gradbeschränkung werden folgende Bedingungen hinzugefügt: TODO anpassen wie zwischen OrTools und Gurobi add/addConstr dargestellt wird

$$\text{model.(add | addConstr)}(\sum x_{e_i} = \deg^*(i))$$

Wobei i der Knoten ist und e_i alle Kanten für diesen Knoten.

Für die *all_edges* Beschränkung werden folgende Bedingungen hinzugefügt:

$$\text{model.(add | addConstr)}((\sum_{j \in I(e)} x_j) + x_e \leq 1)$$

Für die *fix_hull* Beschränkung werden die Kanten der konvexen Hülle fest gesetzt in dem die dazugehörige Variable auf *eins* gesetzt wird. Equivaltend werden für *exclude_edges* und die zuvor berechneten Kanten die Variablen auf *null* gesetzt.

PySat

Da PySat eine SAT Formulierung verwendet müssen die Bedingungen etwas angepasst werden. Die Überschneidungsbeschränkung kann sehr ähnlich mit folgender Formel umgesetzt werden:

$$\neg x_{e_1} \vee \neg x_{e_2} \quad \forall e_1, e_2 \in E : e_2 \in I(e_1)$$

Für die Gradbeschränkung ist das Problem das bei PySat für eine Menge von Variablen nicht festgelegt werden kann wie viele von ihnen wahr sein müssen. Dies kann allerdings mit folgendem Verfahren gelöst werden. TODO rausfinden wie das geht

Die Beschränkungen *fix_hull*, *exclude_edges* und die vorab berechnen Kanten können analog wie in OrTools und Gurobi umgesetzt werden. Hierbei werden anstatt die Variablen auf *null* und *eins* zu setzen, die Klauseln $(\neg x_e)$ und (x_e) hinzugefügt.

Für die *all_edges* Beschränkung kann folgende Klauseln genutzt werden:

$$x_e \vee \bigvee_{e_j \in I(e)} x_{e_j} \quad \forall e \in E$$

3.2.3 Dreieck Formulierung

- Alle möglichen Dreiecke erstellen
- jedes Dreieck als Bool Variable
- Überschneidung können sich überschneidene Dreiecke verwendet werden
 - Sehr langsam und schwer zu berechnen
- Für grad können Sollgrad gleich dreieck anzahl und hülle sollgrad minus eins
 - vlt zeigen an einem Bild
- exclude edges dreiecke verbieten welche exclude edges nutzen

OrTools/Gurobi

- implementiert äquivalenzen zu Kanten
- TODO kann man genau noch mal erklären gucken ob relevant weil Dreiecke schlecht wirken

3.3 Heuristiken

- eine Triangulation erstellen
- den sollGrad möglichst richtig
- Werden mit folgender Formel bewertet

$$\text{diff}_i = \left| \deg^*(i) - \deg(i) \right| \quad (3.4)$$

$$e_i = \begin{cases} \frac{\deg^*(i) - \text{diff}_i}{\deg^*(i)} & \deg^*(i) \geq \text{diff}_i \\ 0 & \text{sonst} \end{cases} \quad (3.5)$$

$$Q_T = \sum_{i=0}^n \frac{e_i}{n} \quad (3.6)$$

Die Formel liefert eine Evaluation (Q_T) für einen $\text{Triangulation}(T)$ mit n Knoten. Hierbei gibt $\deg^*(i)$ den gewünschten Grad an der Stelle i an. Dazu passend gibt $\deg(i)$ den aktuellen Grad an. In der Berechnung (3.4) wird die Difference zwischen dem gewünschten und aktuellen Grad berechnet. In (3.5) wird diese Abweichung Prozentual bewertet. Am Ende wird in (3.6) ein Durchschnitt über alle Prozentualen Werte gebildet.

3.3.1 Flips

In dieser Heuristik werden die in Definition 2.5 definierten Kantenflips verwendet. Ziel ist es, die Gradbeschränkung zu erreichen. Dabei wird ausgenutzt, dass Flips den Grad eines Knotens verändern können die Triangulationseigenschaft dabei aber erhalten bleibt.

Die Heuristik beginnt mit einer initialen Triangulation. Diese wird durch ein klassisches Triangulationsverfahren erzeugt. Anschließend werden maximal k Flips durchgeführt. Ziel ist es, die Gradbeschränkung zu erfüllen.

Zunächst wird mit dem Algorithmus algorithm 1 eine Kante ausgewählt. Diese soll geflippt werden. Der Algorithmus versucht, eine Kante zu finden, deren Knoten nicht den Sollgrad erfüllen. Zusätzlich wird über den Parameter p eine Wahrscheinlichkeit eingeführt. Dadurch kann auch eine Kante ausgewählt werden, deren Knoten den Sollgrad erfüllen. Dies verhindert, dass die Heuristik in einem lokalen Optimum stecken bleibt. Die ausgewählte Kante wird anschließend wie in Abschnitt 3.1.1 beschrieben geflippt. Nach jedem Flip wird überprüft, ob die Gradbeschränkung für alle Knoten erfüllt ist. Sollte dies der Fall sein, wird die Heuristik beendet.

Algorithm 1 CHOOSE EDGE(v, p)

```

1:  $E \leftarrow \{e \mid e \text{ ist ausgehende Kante von } v\}$ 
2: while  $E \neq \emptyset$  do
3:    $e \leftarrow$  zufällige Kante aus  $E$ 
4:    $E \leftarrow E \setminus \{e\}$ 
5:    $u \leftarrow$  Knoten von  $e$  and  $v \neq u$ 
6:   if  $\deg(u) \neq \deg^*(u)$  then
7:     return  $e$ 
8:   else if zu  $p$  % then
9:     return  $e$ 
10:  end if
11: end while
12: return  $e$ 

```

3.3.2 OrTools mit Zielfunktion

Bei dieser Heuristik wird der OrTools-Solver von Google verwendet. Der Vorteil dieses Solvers ist, dass nur wenige Beschränkungen vorgegeben werden müssen. Der Solver sucht dann eigenständig eine Lösung. In diesem Fall wird eine Überschneidungsbeschränkung vorgegeben. Zusätzlich wird eine Zielfunktion definiert, welche die Evaluation (3.6) maximiert.

Im ersten Schritt werden die Kantenvariablen definiert. Für jede Kante e wird eine boolesche Variable x_e erstellt. Für die Überschneidungsbeschränkung wird die Funktion `add_at_most_one` von OrTools verwendet. Diese Funktion wird mit den folgenden Variablenpaaren aufgerufen.

$$(x_{e_1}, x_{e_2}) \quad \forall e_1, e_2 \in E. e_2 \in I(e_1)$$

3.4 Theoretische Mehrere Lösungen

Für die Zielfunktion wird eine Funktion erstellt, die der Gleichung (3.6) ähnelt. Die Teiler in (3.6) und (3.5) werden entfernt. Diese sind für die Maximierung nicht relevant. Da OrTools keine Betragsfunktion in Formeln erlaubt, muss die Funktion *AddAbsEquality* verwendet werden. Für diese Funktion wird eine Integer-Hilfsvariable benötigt. Diese speichert den Betrag der Differenz aus (3.4). Eine Fallunterscheidung für (3.5) ist ebenfalls nicht möglich. Die Variablen e_i werden daher mit folgender Formel berechnet.

$$e_i = \deg^*(i) - \min(\text{diff}_i, \deg^*(i))$$

OrTools erlaubt keine *min*-Funktion in Formeln. Deshalb wird die Funktion *AddMinEquality* verwendet. Auch hierfür wird eine Integer-Hilfsvariable benötigt, welche das Minimum speichert. Diese überarbeitete Formel wird in die Zielfunktion zur Maximierung eingefügt.

TODO glaube das ist einfach nur abs das ist glaube nicht gleichwertig. Es gibt auch eine zweite Zielfunktion welche weniger Integer-Hilfsvariablen verwendet. Für jeden Knoten wird bei dieser Zielfunktion nur eine Integer-Hilfsvariable j_i und keine besonderen Funktionen verwendet. Für jeden Knoten i muss die Hilfsvariable j_i nur folgende Bedingung erfüllen:

$$\begin{aligned} j_i &\geq \deg^*(i) - \deg(i) \\ j_i &\geq \deg(i) - \deg^*(i) \end{aligned}$$

Die Summe aus j_i wird dann als Zielfunktion maximiert.

3.4 Theoretische Mehrere Lösungen

TODO Wo soll das hin

- 3 und 4 Knoten ist Gradbeschränkte Triangulation immer eindeutig
- ab 8 bis unendlich gibt es mehrere Lösungen
- instaze zeigen und erklären wie mehr Punkt hinzufügen

4 Auswertung

4.1 Versuchsumgebung

- Implementation details (e.g., libraries, third-party programs, ...)
- In which environment are you testing (e.g., system specifications)
- What are you testing? (e.g., runtime, space needed, fps, ...)
- What are the results? Show fancy figures!
- What does the results mean? (e.g., give a little discussion on your results)
- ...

4.2 Grund Solver (Eval1)

4.2.1 Dreiecke

Args Legend Mapping (by Solver):

```
=====
gurobi`tri:
—#1 in 300s:
——intersection: True
——degree: True
——exclude`edges: False
—#2 in 300s:
——intersection: True
——degree: True
——exclude`edges: True
OrTools`tri:
—#1 in 300s:
——intersection: True
——degree: True
——exclude`edges: False
—#2 in 300s:
——intersection: True
——degree: True
——exclude`edges: True
```

4 Auswertung

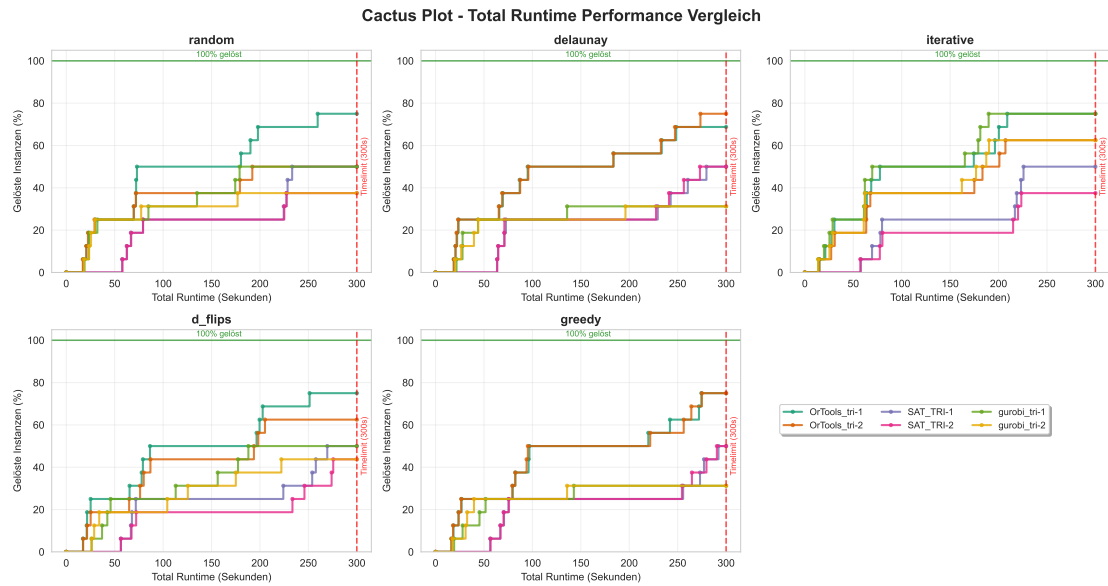


Abbildung 4.1: Enter Caption

SAT TRI:

—#1 in 300s:

—solver`name: glucose3

—add`allEdges`or`exclud`edges: True

—number`tri: False

—intersection: True

—degree: True

—exclude`edges: False

—#2 in 300s:

—solver`name: glucose3

—add`allEdges`or`exclud`edges: True

—number`tri: False

—intersection: True

—degree: True

—exclude`edges: True

=====

4.2.2 Sat

Args Legend Mapping (by Solver):

=====

SAT:

—#1 in 300s:

4.2 Grund Solver (Eval1)

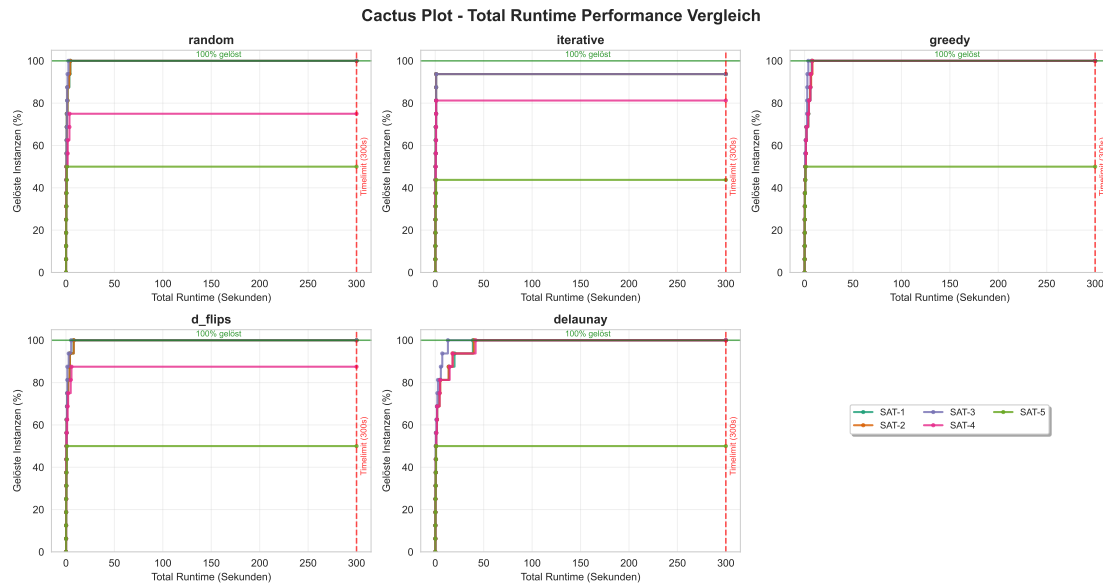


Abbildung 4.2: Enter Caption

```

——solver`name: Glucose42
——add`allEdges`or`exclude`edges: True
——number`edges: False
——intersection: True
——all`edges: False
——degree`exact: True
——degree`atleast: False
——degree`encoding: 1
——degree`subset: False
——fix`hull: False
——exclude`edges: False
——hack`eval6: False
——fix`edges: False
——#2 in 300s:
——solver`name: Glucose42
——add`allEdges`or`exclude`edges: True
——number`edges: False
——intersection: True
——all`edges: False
——degree`exact: True
——degree`atleast: False
——degree`encoding: 1

```

4 Auswertung

——degree`subset: False
——fix`hull: True
——exclude`edges: False
——hack`eval6: False
——fix`edges: False
——#3 in 300s:
——solver`name: Glucose42
——add`allEdges`or`exclude`edges: True
——number`edges: False
——intersection: True
——all`edges: True
——degree`exact: True
——degree`atleast: False
——degree`encoding: 1
——degree`subset: False
——fix`hull: False
——exclude`edges: False
——hack`eval6: False
——fix`edges: False
——#4 in 300s:
——solver`name: Glucose42
——add`allEdges`or`exclude`edges: True
——number`edges: False
——intersection: True
——all`edges: False
——degree`exact: True
——degree`atleast: False
——degree`encoding: 1
——degree`subset: False
——fix`hull: False
——exclude`edges: True
——hack`eval6: False
——fix`edges: False
——#5 in 300s:
——solver`name: Glucose42
——add`allEdges`or`exclude`edges: True
——number`edges: False
——intersection: True
——all`edges: False
——degree`exact: True
——degree`atleast: False
——degree`encoding: 1
——degree`subset: False

4.2 Grund Solver (Eval1)

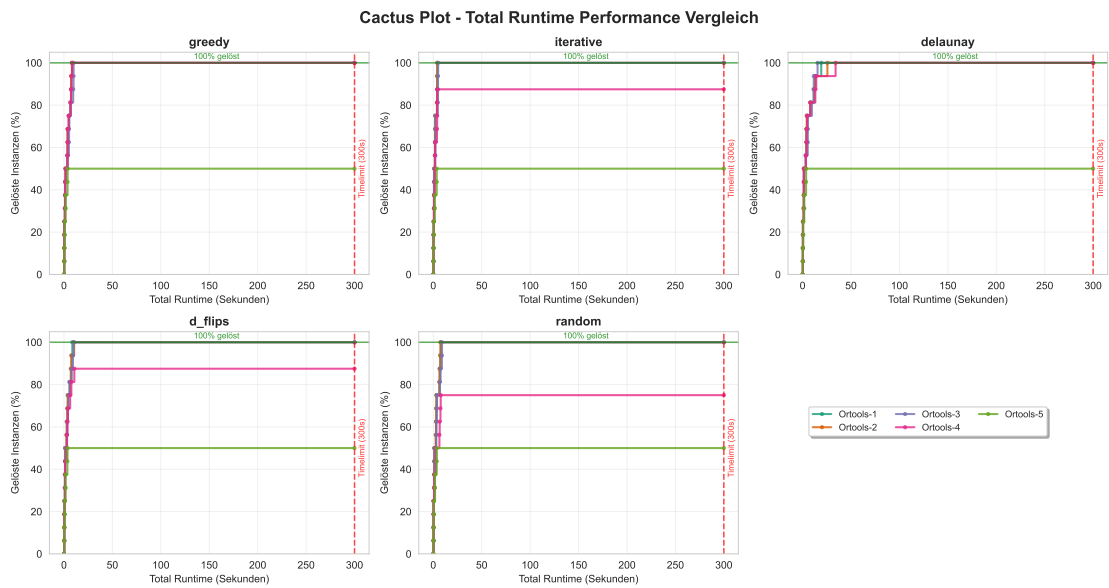


Abbildung 4.3: Enter Caption

—fix`hull: False
—exclude`edges: False
—hack`eval6: False
—fix`edges: True
=====

4.2.3 OrTools

Args Legend Mapping (by Solver):

=====

Ortools:

—#1 in 300s:
—intersection: True
—all`edges: False
—degree: True
—fix`hull: False
—number`edges: False
—evaluation`direction: False
—degree`direction: False
—maximize`edges: False
—exclude`edges: False
—fix`edges: False
—#2 in 300s:

4 Auswertung

```
——intersection: True
——all`edges: False
——degree: True
——fix`hull: True
——number`edges: False
——evaluation`direction: False
——degree`direction: False
——maximize`edges: False
——exclude`edges: False
——fix`edges: False
—#3 in 300s:
——intersection: True
——all`edges: True
——degree: True
——fix`hull: False
——number`edges: False
——evaluation`direction: False
——degree`direction: False
——maximize`edges: False
——exclude`edges: False
——fix`edges: False
—#4 in 300s:
——intersection: True
——all`edges: False
——degree: True
——fix`hull: False
——number`edges: False
——evaluation`direction: False
——degree`direction: False
——maximize`edges: False
——exclude`edges: True
——fix`edges: False
—#5 in 300s:
——intersection: True
——all`edges: False
——degree: True
——fix`hull: False
——number`edges: False
——evaluation`direction: False
——degree`direction: False
——maximize`edges: False
——exclude`edges: False
——fix`edges: True
```


4.2 Grund Solver (Eval1)

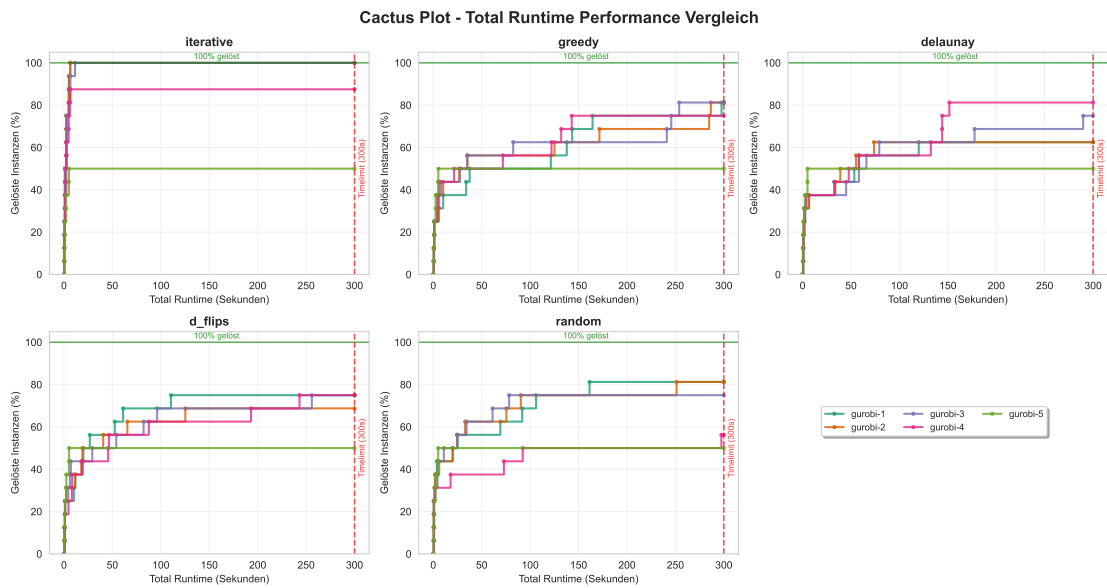


Abbildung 4.4: Enter Caption

4.2.4 Gurobi

Args Legend Mapping (by Solver):

- gurobi:
- #1 in 300s:
 - intersection: True
 - degree: True
 - exclude`edges: False
 - fix`hull: False
 - all`edges: False
 - fix`edges: False
 - #2 in 300s:
 - intersection: True
 - degree: True
 - exclude`edges: False
 - fix`hull: True
 - all`edges: False
 - fix`edges: False
 - #3 in 300s:
 - intersection: True

4 Auswertung

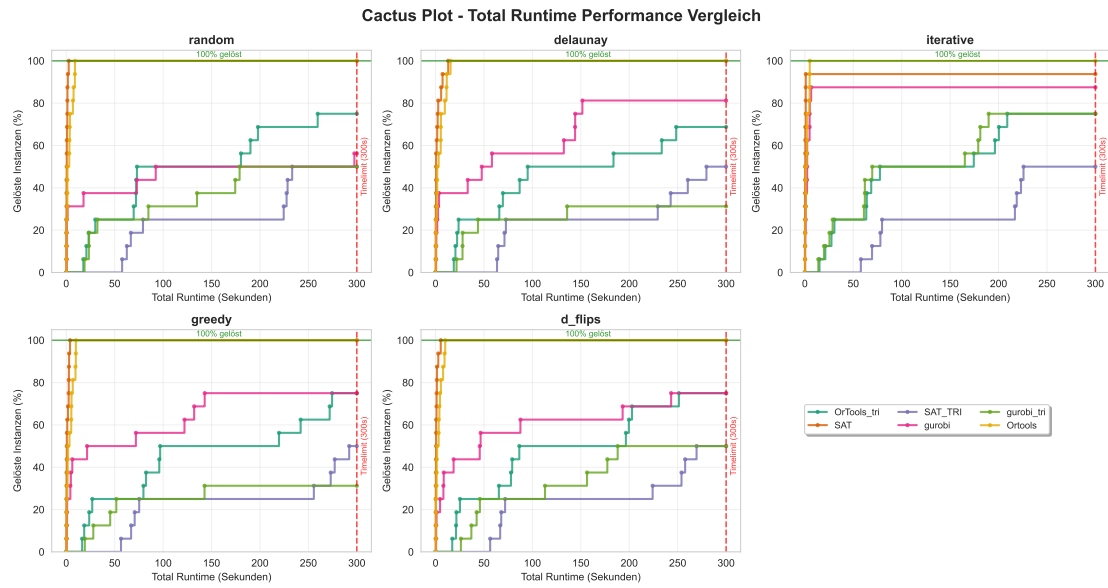


Abbildung 4.5: Enter Caption

—degree: True
 —exclude`edges: False
 —fix`hull: False
 —all`edges: True
 —fix`edges: False
 —#4 in 300s:
 —intersection: True
 —degree: True
 —exclude`edges: True
 —fix`hull: False
 —all`edges: False
 —fix`edges: False
 —#5 in 300s:
 —intersection: True
 —degree: True
 —exclude`edges: False
 —fix`hull: False
 —all`edges: False
 —fix`edges: True

4.2.5 Gesamt

Args Legend Mapping (by Solver):

```
=====
SAT:
—#1 in 300s:
——solver`name: Glucose42
——add`allEdges`or`exclude`edges: True
——number`edges: False
——intersection: True
——all`edges: True
——degree`exact: True
——degree`atleast: False
——degree`encoding: 1
——degree`subset: False
——fix`hull: False
——exclude`edges: False
——hack`eval6: False
——fix`edges: False
gurobi:
—#1 in 300s:
——intersection: True
——degree: True
——exclude`edges: True
——fix`hull: False
——all`edges: False
——fix`edges: False
Ortools:
—#1 in 300s:
——intersection: True
——all`edges: True
——degree: True
——fix`hull: False
——number`edges: False
——evaluation`direction: False
——degree`direction: False
——maximize`edges: False
——exclude`edges: False
——fix`edges: False
gurobi`tri:
—#1 in 300s:
——intersection: True
——degree: True
——exclude`edges: False
```

4 Auswertung

OrTools'tri:

—#1 in 300s:

——intersection: True

——degree: True

——exclude'edges: False

SAT'TRI:

—#1 in 300s:

——solver'name: glucose3

——add'allEdges'or'exlucde'edges: True

——number'tri: False

——intersection: True

——degree: True

——exclude'edges: False

=====

5 Conclusion (and Future Work)

This is the end of your thesis! Give a summary of what you did.

You can also write down possible future work you or other people could look at.

- meine Random instanzen waren nicht normalverteilt
- Könnte man in zukunft anschauen
- gibt Paper mit einfachen Polygone mit normalverteilter Triangulation

Literaturverzeichnis

- [1] Prosenjit Bose and Ferran Hurtado. Flips in planar graphs. *Computational Geometry*, 42(1):60–80, 2009. URL: <https://www.sciencedirect.com/science/article/pii/S0925772108000370>, doi:10.1016/j.comgeo.2008.04.001.
- [2] Basudeb Datta and Subhojoy Gupta. Degree-regular triangulations of surfaces. *arXiv preprint arXiv:1711.01247*, 2017.
- [3] Ana Maria de Almeida, Pedro Martins, and Maurício C de Souza. Min-degree constrained minimum spanning tree problem: complexity, properties, and formulations. *International Transactions in Operational Research*, 19(3):323–352, 2012.
- [4] Mark de Berg, Marc van Kreveld, Mark Overmars, and Otfried Cheong Schwarzkopf. *Delaunay Triangulations*, pages 183–210. Springer Berlin Heidelberg, Berlin, Heidelberg, 2000. doi:10.1007/978-3-662-04245-8_9.
- [5] Sándor P Fekete, Phillip Keldenich, and Michael Perk. Exact algorithms for minimum dilation triangulation. *arXiv preprint arXiv:2502.18189*, 2025.
- [6] Sándor P Fekete, Samir Khuller, Monika Klemmstein, Balaji Raghavachari, and Neal Young. A network-flow technique for finding low-weight bounded-degree spanning trees. *Journal of Algorithms*, 24(2):310–324, 1997.
- [7] Laxmi P Gewali and Bhaikaji Gurung. Degree aware triangulation of annular regions. In *Information Technology-New Generations: 15th International Conference on Information Technology*, pages 683–690. Springer, 2018.
- [8] F. Hurtado, M. Noy, and J. Urrutia. Flipping edges in triangulations. In *Proceedings of the Twelfth Annual Symposium on Computational Geometry, SCG '96*, page 214–223, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/237218.237367.
- [9] J. Knowles and D. Corne. A new evolutionary approach to the degree-constrained minimum spanning tree problem. *IEEE Transactions on Evolutionary Computation*, 4(2):125–134, 2000. doi:10.1109/4235.850653.
- [10] Mohan Krishnamoorthy, Andreas T. Ernst, and Yazid M. Sharaiha. Comparison of algorithms for the degree constrained minimum spanning tree. *Journal of Heuristics*, 7(6):587–611, 2001. doi:10.1023/A:1011977126230.
- [11] Oleg R Musin. Properties of the delaunay triangulation. In *Proceedings of the thirteenth annual symposium on Computational geometry*, pages 424–426, 1997.

Literaturverzeichnis

- [12] Micha Sharir, Adam Sheffer, and Emo Welzl. On degrees in random triangulations of point sets. In *Proceedings of the twenty-sixth annual symposium on Computational geometry*, pages 297–306, 2010.